

Omnibase Docs

Developers

- Rogelio Ruiz : joserogelioruiz12@outlook.com

Current State of Project

asdfasdfasdfasdf

To do:

- Poner comentarios con que ID es cada motor FL, BL, FR, BR
- Implementar máquina de estados para que no se tenga que enviar y recibir absolutamente todos los valores (si no son muchos aun así jala bien pero pa q sea más mamón)??
- Hacer que BT_active sea 1 si está conectado y 2 si está activo para verlo en dashboard
- Hacer que la esp mande un heartbeat para corroborar que sigue conectado

Index

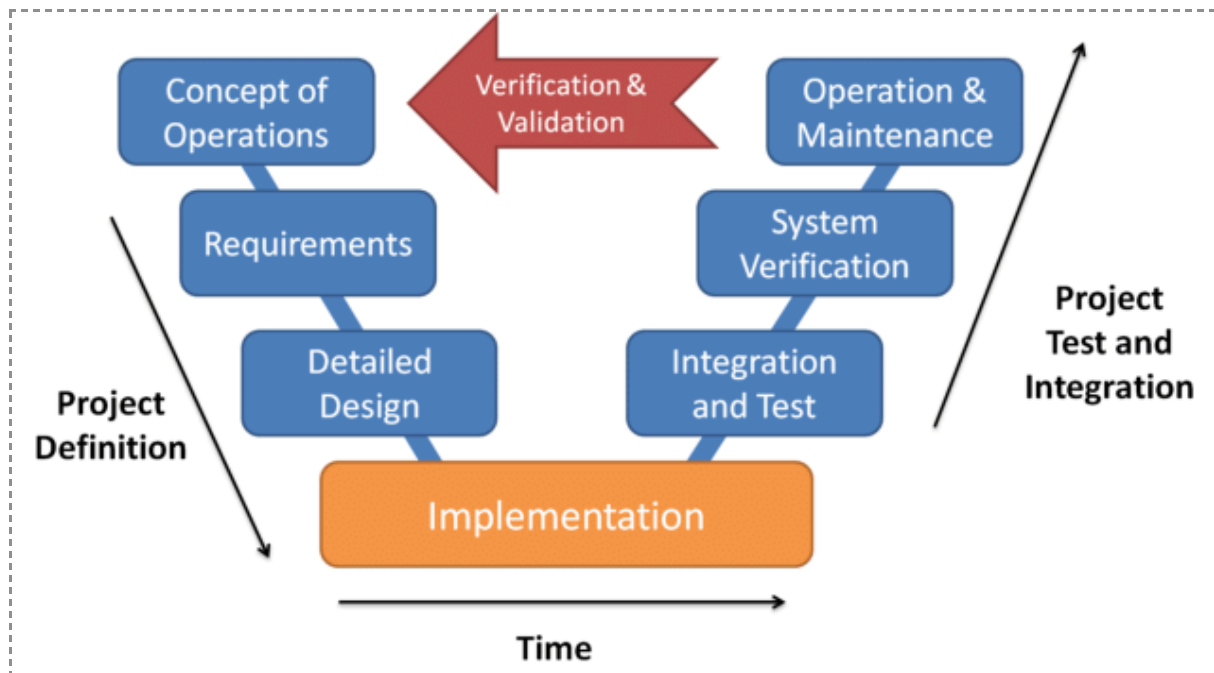
To do:.....	1
Index.....	2
PINOUT.....	3
REQUERIMIENTOS DEL SISTEMA.....	4
REQUERIMIENTOS DEL HARDWARE.....	5
TUTORIAL.....	6
ROS2 (PENDIENTE).....	7
Odrive.....	9
Descripción.....	9
Configuración.....	9
Configuración actual.....	9
Odrive CAN.....	11
CANABLE.....	12
MCP2515.....	13
FD CAN (with TJA1051 PENDIENTE PROBAR).....	14
RTOS.....	18
ODriveCanTask.....	18
UartRxTask.....	18
UartTxTask.....	18
ROS2 SerialComm Node.....	18
Code Block Diagram (PENDIENTE).....	19
BNO055.....	22
Kinematics (PENDIENTE REDACTAR LIVESCRIPT).....	23
Full Book MODERN ROBOTICS: MECHANICS, PLANNING, AND CONTROL..	
23	
Analysis (PENDIENTE REDACTAR BIEN LO HECHO EN MATLAB).....	23
Memory usage investigation.....	25
Dualshock4 PS4 Bluetooth connection.....	26
Troubleshooting.....	27
Resources.....	28
Núcleo STM32H755 Documentation.....	28
SCANF y PRINTF.....	28
System Architecture.....	29
Motor con encoder en Chaneque.....	29
Female Headers Pinout.....	30
UART Configuration.....	31
Uart polling and interrupt.....	31
Clock Configuration.....	32
TIMERS.....	33
PWM.....	33
Encoder mode.....	35

PINOUT

PIN	Función
PB6	I2C_SCL (BNO055)
PB7	I2C_SDA (BNO055)
PD5	UART2_TX (ESP32)
PD6	UART2_RX (ESP32)
PD0	FDCAN1_RX (TJA1051)
PD1	FDCAN1_TX (TJA1051)
PD8 (USB port)	UART2_TX
PD9 (USB port)	UART2_RX

REQUERIMIENTOS DEL SISTEMA

Por Deivideich



Requerimientos del sistema

- Debe tener un MCU (STM32HX o *similar*)
- Debe tener un MCU auxiliar para comunicación inalámbrica (ej. ESP32)
 - Alternativamente un módulo de comunicación inalámbrica
 - Ej. Módulo RF24
- Todos los periféricos se conectan a esta placa. (XT60, salidas de drivers, entradas de alimentación, cables ethernet, comunicación)
- Botones de reset y otros básicos para debug
- Serial para comunicarse con la computadora principal, a decisión del diseño, UART, RJ45, etc.
- Sensores integrados
 - Ultrasónicos
 - Conexión a motores (drivers)
 - BNO/IMU
 - LIDAR (canal de coms)
 - Encoders
 - Mínimo dos bumpers
 - Al menos una entrada para botón (user button)
 - Mini fusibles para la STM
 - Revisar con la especificación de la protección integrada de overcurrent de STM
 - Polyfuse
- Salida de información por medio sonoro (buzzer)
- Monitoreo de otras placas:
 - Sensado de corriente global
 - Temperatura de componentes selectos

- Pines de comunicaciones
 - GPIO
 - Protocolos de comunicación
- STLink
- ROHS

REQUERIMIENTOS DEL HARDWARE

Por David H

Requerimientos de hardware

- Uso de STM32H755ZIT6 para realizar control de los motores
- ESP32 para comunicación inalámbrica
- Micro-usb para comunicación Compu – Placa
- IMU
- Probar interfaz ethernet
- Conectores JST o Molex-470531000
- Botones NA para user button y reset
- Con la compu principal aún está pendiente comparar Micro-USB vs Ethernet
- Ultrasónicos
 - HC-04
- Conexiones a motores a través de interfaz CAN
- Drivers Odrive S1
 - Comunicación a través de FDCAN de stm32h7
 - Interfaz CAN MCP2551 (Como alternativa al FDCAN)

TUTORIAL

- Connect STM32H7 to Computer (laptop, rpi, jetson)
- If not already there download the github repo
<https://github.com/RoBorregos/home-custom-base/tree/firmware>
- Within the repo go to the omnibase_ws folder, build it, and source it
- run the following command
`ros2 run odrive_comm odrive_dashboard`
- The dashboard should be available on:
<http://localhost:5000/offline>

ROS2 (PENDIENTE)

```
roger@JRPC: ~/Github/home-electronics/omnibase_ws
# roger@JRPC: ~/Github/home-electronics/omnibase_ws 74x14
about parameters: asynchronous service call failed: the target node may not be spinning
[WARN] [1756357508.358403838] [rqt_reconfigure]: Failed to get information about parameters: asynchronous service call failed: the target node may not be spinning
[WARN] [1756357509.362934204] [rqt_reconfigure]: Failed to get information about parameters: asynchronous service call failed: the target node may not be spinning
^C[INFO] [1756357511.025696892] [rclcpp]: signal_handler(signum=2)
^[[Arroger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 run rqt_reconfigure rqt_reconfigure
^C[INFO] [1756357770.146642056] [rclcpp]: signal_handler(signum=2)
roger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 run rqt_reconfigure rqt_reconfigure
roger@JRPC:~/Github/home-electronics/omnibase_ws 150x14
ODOM_Err_phi=-2.333,U_Err_1=0.000,U_Err_2=0.000,U_Err_3=0.000,U_Err_4=0.000,Ctrl_Inertial_x_dot=4.000,Ctrl_Inertial_y_dot=6.000,Ctrl_Inertial_phi_dot=0.000,Ctrl_necc_u1=0.000,Ctrl_necc_u2=0.000,Ctrl_necc_u3=0.000,Ctrl_necc_u4=0.000,ts_current=23712,ts_previous=23612,ts_delta=100,Ctrl_duty_u1=13841,Ctrl_duty_u2=9175,Ctrl_duty_u3=0,Ctrl_duty_u4=0,xKp=10.000,xKl=0.100,xKd=0.000,yKp=10.000,yKl=0.100,yKd=0.000,phiKp=0.500,phiKl=0.100,phiKd=0.000,u0Kp=30.500,u0Kl=0.500,u0Kd=1.010,u1Kp=30.500,u1Kl=0.500,u1Kd=1.010,u2Kp=30.500,u2Kl=0.500,u2Kd=1.010,u3Kp=30.5...
data: x_desired=-6.000000,y_desired=-3.000000,phi_desired=0.000000,d=1.000000,r=1.000000,roll=-1.437500,pitch=9.687500,yaw=359.937500,TIM1=11738,TIM2=57755,TIM4=0,TIM8=0,Enc_Wheel_Omega1=0.000000,Enc_Wheel_Omega2=0.000000,Enc_Wheel_Omega3=0.000000,Enc_Wheel_Omega4=0.000000,Inertial_ang_vel_calc=0.000000,Inertial_x_vel_calc=0.000000,Inertial_y_vel_calc=0.000000,ODOM_phi=2.333,ODOM_x_pos=-15.629,ODOM_y_pos=3.918,ODOM_Err_x=9.629,ODOM_Err_y=-6.918,ODOM_Err_phi=-2.333,U_Err_1=0.000,U_Err_2=0.000,U_Err_3=0.000,U_Err_4=0.000,Ctrl_Inertial_x_dot=4.000,Ctrl_Inertial_y_dot=6.000,Ctrl_Inertial_phi_dot=0.000,Ctrl_necc_u1=0.000,Ctrl_necc_u2=0.000,Ctrl_necc_u3=0.000,Ctrl_necc_u4=0.000,ts_current=23817,ts_previous=23712,ts_delta=105,Ctrl_duty_u1=13841,Ctrl_duty_u2=9175,Ctrl_duty_u3=0,Ctrl_duty_u4=0,xKp=10.000,xKl=0.100,xKd=0.000,yKp=10.000,yKl=0.100,yKd=0.000,phiKp=0.500,phiKl=0.100,phiKd=0.000,u0Kp=30.500,u0Kl=0.500,u0Kd=1.010,u1Kp=30.500,u1Kl=0.500,u1Kd=1.010,u2Kp=30.500,u2Kl=0.500,u2Kd=1.010,u3Kp=30.5...
^Croger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 topic echo /stm32/raw --truncate-length 1000
```

```
roger@JRPC: ~
# roger@JRPC: ~ 29x24
- 57755
- 0
- 0
---
layout:
  dim: []
  data_offset: 0
data:
- 11738
- 57755
- 0
- 0
---
layout:
  dim: []
  data_offset: 0
data:
- 11738
- 57755
- 0
- 0
---
^Croger@JRPC:~$ ros2 topic echo /stm32/encoders

# roger@JRPC: ~ 29x24
- 0.0
- 0.0
- 0.0
---
layout:
  dim: []
  data_offset: 0
data:
- 0.0
- 0.0
- 0.0
- 0.0
---
layout:
  dim: []
  data_offset: 0
data:
- 0.0
- 0.0
- 0.0
- 0.0
---
^Croger@JRPC:~$ ros2 topic echo /stm32/omegas

# roger@JRPC: ~ 29x24
- 9175
- 0
- 0
---
layout:
  dim: []
  data_offset: 0
data:
- 13841
- 9175
- 0
- 0
---
layout:
  dim: []
  data_offset: 0
data:
- 13841
- 9175
- 0
- 0
---
^Croger@JRPC:~$ ros2 topic echo /stm32/pwm

# roger@JRPC: ~ 29x24
- 0.0
- 0.0
- 0.0
---
layout:
  dim: []
  data_offset: 0
data:
- 0.0
- 0.0
- 0.0
- 0.0
---
layout:
  dim: []
  data_offset: 0
data:
- 0.0
- 0.0
- 0.0
- 0.0
---
^Croger@JRPC:~$ ros2 topic echo /stm32/u_errors
```

```
roger@JRPC: ~/Github/home-electronics/omnibase_ws
File "/home/roger/Github/home-electronics/omnibase_ws/install/serial_comm/lib/python3.10/site-packages/serial_comm/serial_communication.py", line 120, in send_message
    self.ser.write(cmd_str.encode())
File "/home/roger/.local/lib/python3.10/site-packages/serial/serial_posix.py", line 655, in write
    raise SerialException('write failed: {}'.format(e))
serial.SerialUtil.SerialException: write failed: [Errno 19] No such device
[ros2run]: Process exited with failure 1
roger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 run serial_comm serial_communication
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x12
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x5
ros2 run rqt_reconf figure_rqt_reconfigure
^C[INFO] [1748399337.788836234] [rclcpp]: signal_handler(signum=2)
roger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 run rqt_reconf figure_rqt_reconfigure
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x5
ros2 run rqt_plot rqt_plot
^C[INFO] [1748399336.586521221] [rclcpp]: signal_handler(signum=2)
roger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 run rqt_plot rqt_plot
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x12
layout:
dim: []
data_offset: 0
data:
- 1409
- 4336
- 6030
- 2336
...
^Croger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 topic echo /stm32/pwm
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x5
layout:
dim: []
data_offset: 0
data:
- 1.0
- 4.599999904632568
- 3.5
- 2.0
...
^Croger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 topic echo /stm32/ctrl_u
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x5
layout:
dim: []
data_offset: 0
data:
- 0.010735999792814255
- 0.05525999888777733
- 0.04366699978709221
- 0.032758999961590767
...
^Croger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 topic echo /stm32/omegas
```

```
roger@JRPC:~/Github/home-electronics/omnibase_ws 69x5
layout:
dim: []
data_offset: 0
data:
- 257
- 59980
- 4353
- 53316
...
^Croger@JRPC:~/Github/home-electronics/omnibase_ws$ ros2 topic echo /stm32/encoders
```

Odrive

Descripción

- Pay to win de control de brushless motor
- Se usa el modelo ODrive S1 ([datasheet](#))
- Encoder incluido
- 12 V DC **IMPORTANTE: POLARIDAD NO INTERCAMBIABLE**

Configuración

Para configurar el Odrive conectarlo a una computadora y configurarlo desde la GUI

<https://gui.odriverobotics.com/configuration>:

(Nota: en teoría también se puede a través de CAN)

Troubleshooting: <https://docs.odriverobotics.com/v/latest/interfaces/gui.html>

Configuración actual

The screenshot shows the ODrive GUI configuration interface. The main window is titled 'Configuring ODrive 0' and has four tabs: 1 Power Source, 2 Motor, 3 Encoder, and 4 Control mode. The 'Power Source' tab is active, showing settings for DC bus overvoltage trip level (29 V), Internal current limit (80 A), DC bus undervoltage trip level (10.5 V), DC max positive current (15 A), DC max negative current (-0.01 A), and Use brake resistor (2 Ω). A warning box states: 'These parameters are trip levels, not soft limits. That means the ODrive will not actively try to stay within these limits, but it will go to IDLE and report an error if the limits are exceeded.' Below the tabs, there are six motor options: D5065-270KV, D6374-150KV, D5312s-330KV, M8325s-100KV, BotWheel, and N3401-AMT21. A blue button labeled 'Other Motor' is also present. The 'Motor Parameters' section on the right shows settings for Type (High current), Pole pairs (20), KV (90 rpm/V), KT (0.092 Nm/A), Current limit (10 A), Current limit torque (0.919 Nm), Motor calib. current (10 A), Motor calib. voltage (2 V), Lock-in spin current (10 A), and a checkbox for 'Use thermistor'.

Configuring ODrive 0

1 Power Source 2 Motor 3 Encoder 4 Control mode

Power Source

Configure these settings according to the limits of your power source. [Learn more.](#)

DC bus overvoltage trip level ⓘ 29 V ↺

Internal current limit 80 A

DC bus undervoltage trip level ⓘ 10.5 V ↺

☒ DC max positive current ⓘ 15 A ↺

☒ DC max negative current ⓘ -0.01 A ↺

☒ Use brake resistor ⓘ 2 Ω ↺

ⓘ These parameters are trip levels, not soft limits. That means the ODrive will not actively try to stay within these limits, but it will go to IDLE and report an error if the limits are exceeded.

1 Power Source 2 Motor 3 Encoder 4 Control mode 5 Interfaces 6 Apply & Calibrate

D5065-270KV D6374-150KV D5312s-330KV M8325s-100KV BotWheel N3401-AMT21

Other Motor

Motor Parameters

Type ⓘ High current ↺

Pole pairs ⓘ 20 ↺

KV ⓘ 90 rpm/V ↺

KT 0.092 Nm/A

Current limit ⓘ 10 A ↺

Current limit torque 0.919 Nm

Motor calib. current ⓘ 10 A ↺

Motor calib. voltage ⓘ 2 V ↺

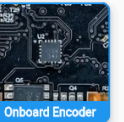
Lock-in spin current ⓘ 10 A ↺

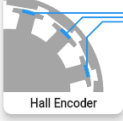
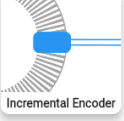
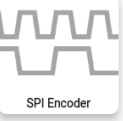
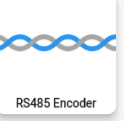
☐ Use thermistor ⓘ

1 Power Source
2 Motor
3 Encoder
4 Control mode
5 Interfaces
6 Apply & Calibrate





Encoder Parameters

Type Onboard encoder

☐ Use separate commutation encoder

This encoder will be used for position control, velocity control and current control ([commutation](#)). To use a different encoder for current control, check the checkbox above.

1 Power Source
2 Motor
3 Encoder
4 Control mode
5 Interfaces
6 Apply & Calibrate

1 **Ramped Velocity Control** is recommended for initial setup and tuning in the Dashboard. The control mode and some parameters on this page can also be changed on the Dashboard.

Control Mode

Control Mode

Ramp limit turns/s²

Soft velocity limit turns/s

Hard velocity limit turns/s

☐ Torque limit Nm

1 Power Source
2 Motor
3 Encoder
4 Control mode
5 Interfaces
6 Apply & Calibrate

USB

USB communication is always enabled without additional configuration. [Learn more.](#)

CAN Bus

☒ **Enable**

CAN Bus is the recommended way to control the ODrive. [Learn more.](#)

Bitrate bits/s

☒ Node ID

☒ Send heartbeat every ms

Reference: [Heartbeat](#)

☒ Send feedback every ms

Reference: [Get_Idx](#)
[Get_Torques](#)
[Get_Error](#)
[Get_Temperature](#)
[Get_Bus_Voltage_Current](#)

UART

☐ Enable

Simple text based interface. Only recommended if CAN is not available. [Learn more.](#)

Watchdog timer

☐ Enable

The watchdog disables the motor when communication is disrupted. [Learn more.](#)

GPIO Input

Use the PWM signal from a remote control or an analog signal to control your ODrive.

The meaning of the inputs depend on your selected control mode. One of the inputs will act as setpoint and the others act as feedforward terms or have no effect.

Position

Velocity

Torque

1 Power Source
2 Motor
3 Encoder
4 Control mode
5 Interfaces
6 Apply & Calibrate

This page guides you through the final setup and calibration steps. To avoid any unexpected behavior, please run all steps in the order they are presented.

- Erase old configuration**
 Ensures that no old configuration is on the device. This is required for the new configuration to be correct.
Erase & Reboot
- Apply new configuration**
 This writes the settings onto the ODrive according to the configuration on the previous pages.
 Python commands (advanced users) ▼
Apply
- Save to non-volatile memory**
 Saving and rebooting the device is necessary for some changes to take effect.
Save & Reboot
- Calibrate**
 This will keep your motor and spin it to calibrate the motor and encoder. **Make sure the motor can move freely.**

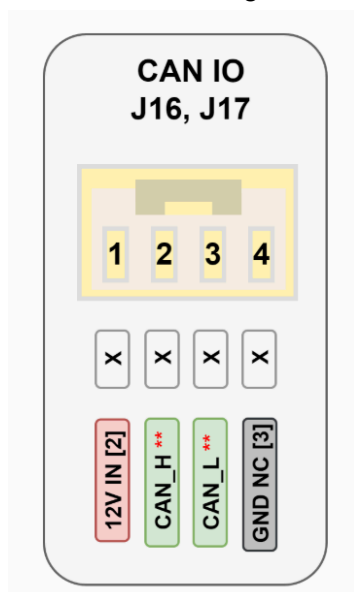
1 Magnetic encoders may exhibit nonlinearities. If you encounter vibration problems at velocities >10rps, consider trying our new [Harmonic Compensation](#). (needs [pre-release](#) firmware, improved calibration flow coming soon)

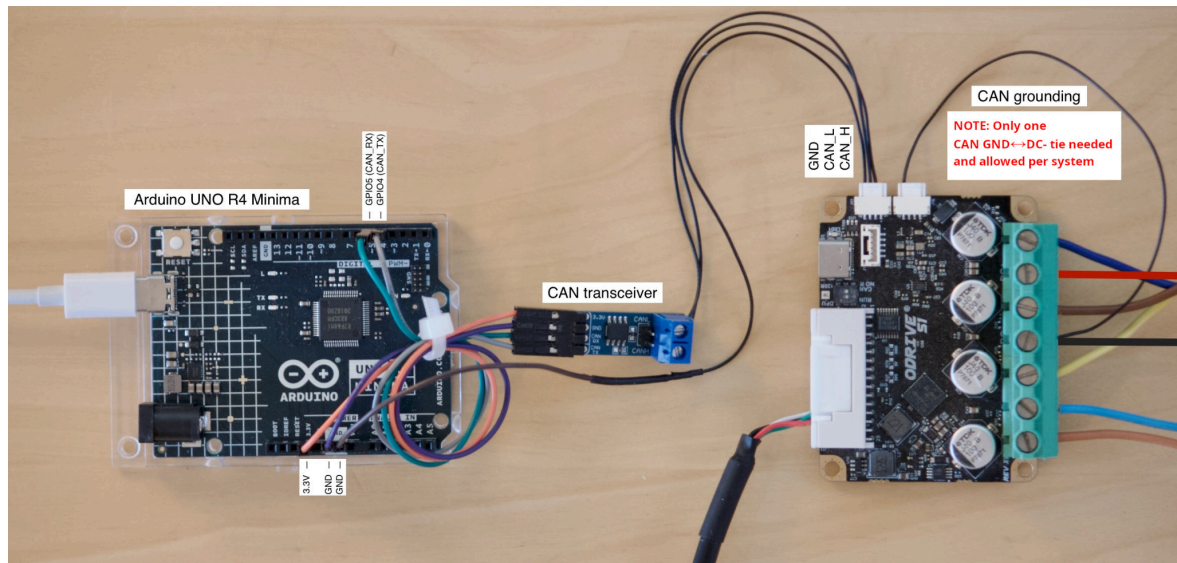
Run Calibration Sequence
- Save to non-volatile memory**
 This ensures that the calibration persists across reboots.
Save & Reboot
- What's next?**
 The ODrive is now ready to spin your motor! Head over to the **Dashboard** tab to try it out! To control and monitor the ODrive from other devices, see [here](#).

Si se cambia la configuración debes elección cada uno de los botones de esta sección en orden. La opción “Run Calibration Sequence” solo es posible al estar conectado a los 12V DC necesarios, consume ~1.3A al calibrar.

Odrive CAN

BITRATE En configuración actual 500kB/s





Para conectar el odribe a CAN es necesario conectar los pines CAN_H y CAN_L a los correspondientes en el bus CAN y GND al GND de los microcontroladores.

Es necesario cambiar el Axis_State según el modo de control (chechar documentación, no entendí bien todavía). Esto se puede hacer desde el gui en línea picándole al botón de power.

Base para implementación de funciones de control/configuración del odribe por CAN:

<https://github.com/siddarthiyer/ODrive-STM32-CAN-Driver/tree/main>

<https://docs.odriverobotics.com/v/latest/manual/can-protocol.html>

<https://docs.odriverobotics.com/v/latest/guides/can-guide.html>

<https://gui.odriverobotics.com/dashboard>

<https://docs.odriverobotics.com/v/latest/guides/odrivetool-setup.html>

<https://docs.odriverobotics.com/v/latest/guides/getting-started.html>

<https://docs.odriverobotics.com/v/latest/interfaces/gui.html>

CANABLE

On linux

Install can-utils package:

```
sudo apt update
```

```
sudo apt install can-utils
```

Check that slcand is installed:

```
dpkg -L can-utils | grep slcand
```

Check what port the canable was connected to:

```
ls /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
```

Enable that port as a can# intergace (-s6 significa 500kbit):

```
sudo slcand -o -c -s6 /dev/ttyACM1 can0
```

```
sudo ip link set up can0 txqueuelen 1000
```

More documentation: <https://canable.io/getting-started.html#cangaroo>

Example Commands:

- Create CAN interface from CANable (serial to SocketCAN):
`sudo slcand -o -c -s6 /dev/ttyACM0 can0`
- Bring the CAN interface up (activate it):
`sudo ifconfig can0 up`
- Increase transmit queue length to avoid dropped frames:
`sudo ifconfig can0 txqueuelen 1000`
- Send a CAN frame with ID 0x999 and payload 0xDEADBEEF:
`cansend can0 999#DEADBEEF`
- Dump all CAN traffic from can0 in real time:
`candump can0`
- Show CAN bus load percentage at 500 kbit/s:
`canbusload can0@500000`
- Sniff CAN traffic in top-style view (grouped by ID, highlights changes):
`cansniffer can0`
- Generate fixed CAN messages with 8-byte payload 11 22 33 44 DE AD BE EF:
`cangen can0 -D 11223344DEADBEEF -L 8`

MCP2515

Driver q falta checar:

<https://github.com/siddarthiyer/ODrive-STM32-CAN-Driver/tree/main>

Librería del MCP2515 usada:

<https://github.com/thumptechnology/STM32-MCP2515?tab=readme-ov-file>

- Se usaron los archivos mcp2515.c, mcp2515.h y mcp2515_consts.h además de incluir el can.h de la librería de arduino.

<https://github.com/autowp/arduino-mcp2515?tab=readme-ov-file>

FD CAN (with TJA1051 PENDIENTE PROBAR)

Basic Parameters	
Frame Format	Classic mode
Mode	Normal mode
Auto Retransmission	Disable
Transmit Pause	Disable
Protocol Exception	Enable
Nominal Sync Jump Width	8
Data Prescaler	1
Data Sync Jump Width	1
Data Time Seg1	1
Data Time Seg2	1
Message Ram Offset	0
Std Filters Nbr	1
Ext Filters Nbr	0
Rx Fifo0 Elmts Nbr	1
Rx Fifo0 Elmt Size	8 bytes data field
Rx Fifo1 Elmts Nbr	0
Rx Fifo1 Elmt Size	8 bytes data field
Rx Buffers Nbr	0
Rx Buffer Size	8 bytes data field
Tx Events Nbr	0
Tx Buffers Nbr	0
Tx Fifo Queue Elmts Nbr	1
Tx Fifo Queue Mode	FIFO mode
Tx Elmt Size	8 bytes data field
Clock Calibration Unit	
Clock Calibration	Disable
Bit Timings Parameters	
Nominal Prescaler	2
Nominal Time Quantum	50.0 ns
Nominal Time Seg1	31
Nominal Time Seg2	8
Nominal Time for one Bit	2000 ns
Nominal Baud Rate	500000 bit/s

▶ FDCAN in STM32 || LoopBack Mode

```
if (HAL_FDCAN_GetRxMessage(&hfdcan1, FDCAN_RX_FIFO0, &RxHeader, RxData) == HAL_OK) {  
    if (RxHeader.IdType == FDCAN_EXTENDED_ID) {  
        printf("Extended ID: 0x%lx\n\r", RxHeader.Identifier);  
    } else {  
        printf("Standard ID: 0x%lx\n\r", RxHeader.Identifier);  
    }  
}
```

```

        dataLength = (RxHeader.DataLength >> 16) & 0x0F; // Extract
DLC (0-8 bytes)
        for (int i = 0; i < (dataLength); i++) {
            printf("0x%02X ", RxData[i]);
        }
        printf("Message received successfully\n\r");
    } else {
        printf("No message received\n\r");
    }

    HAL_Delay(100); // Avoid tight loop
    if (HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan1, &TxHeader, TxData) ==
HAL_OK) {
        printf("Message sent successfully\n\r");
    } else {
        printf("Failed to send message\n\r");
    }

    if (HAL_FDCAN_GetRxMessage(&hfdcan1, FDCAN_RX_FIFO0, &RxHeader,
RxData) == HAL_OK) {
        uint8_t dataLength = (RxHeader.DataLength & 0x0F); // Extract
DLC

        if (dataLength > 0) {
            printf("Received ID: 0x%lx\n\r", RxHeader.Identifier);
            printf("Data: ");
            for (int i = 0; i < dataLength; i++) {
                printf("0x%02X ", RxData[i]);
            }
            printf("\n\r");
        } else {
            printf("Received message with no data.\n\r");
        }
    } else {
        printf("No message received.\n\r");
    }
}

```

```

static void MX_FDCAN1_Init(void)
{
    /* USER CODE BEGIN FDCAN1_Init 0 */
    /* USER CODE END FDCAN1_Init 0 */
    /* USER CODE BEGIN FDCAN1_Init 1 */
    /* USER CODE END FDCAN1_Init 1 */
    hfdcan1.Instance = FDCAN1;
    hfdcan1.Init.FrameFormat = FDCAN_FRAME_CLASSIC;
    hfdcan1.Init.Mode = FDCAN_MODE_NORMAL;
    hfdcan1.Init.AutoRetransmission = DISABLE;
    hfdcan1.Init.TransmitPause = DISABLE;
    hfdcan1.Init.ProtocolException = ENABLE;
    hfdcan1.Init.NominalPrescaler = 2;
    hfdcan1.Init.NominalSyncJumpWidth = 8;
    hfdcan1.Init.NominalTimeSeg1 = 31 ;
    hfdcan1.Init.NominalTimeSeg2 = 8;
    hfdcan1.Init.DataPrescaler = 1;
    hfdcan1.Init.DataSyncJumpWidth = 1;
}

```

```

hfdcan1.Init.DataTimeSeg1 = 1;
hfdcan1.Init.DataTimeSeg2 = 1;
hfdcan1.Init.MessageRAMOffset = 0;
hfdcan1.Init.StdFiltersNbr = 1;
hfdcan1.Init.ExtFiltersNbr = 0;
hfdcan1.Init.RxFifo0ElmtsNbr = 1;
hfdcan1.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_8;
hfdcan1.Init.RxFifo1ElmtsNbr = 0;
hfdcan1.Init.RxFifo1ElmtSize = FDCAN_DATA_BYTES_8;
hfdcan1.Init.RxBuffersNbr = 0;
hfdcan1.Init.RxBufferSize = FDCAN_DATA_BYTES_8;
hfdcan1.Init.TxEventsNbr = 0;
hfdcan1.Init.TxBuffersNbr = 0;
hfdcan1.Init.TxFifoQueueElmtsNbr = 1;
hfdcan1.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION;
hfdcan1.Init.TxElmtSize = FDCAN_DATA_BYTES_8;
if (HAL_FDCAN_Init(&hfdcan1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN FDCAN1_Init 2 */
/*AAO+*/
/* Configure Rx filter */
sFilterConfig.IdType = FDCAN_EXTENDED_ID;           // Set to Extended ID
sFilterConfig.FilterIndex = 0;                       // Filter index
sFilterConfig.FilterType = FDCAN_FILTER_MASK;        // Filter type (Mask
mode)
sFilterConfig.FilterConfig = FDCAN_FILTER_TO_RXFIFO0; // Direct matching
frames to FIFO 0
sFilterConfig.FilterID1 = 0x18FF1220; // Base ID
sFilterConfig.FilterID2 = 0xFFFFFFFF; // Mask to allow only last 2 bits to
vary
/* Configure global filter to reject all non-matching frames */
HAL_FDCAN_ConfigGlobalFilter(&hfdcan1,
    FDCAN_ACCEPT_IN_RX_FIFO0, // Accept standard messages in FIFO0
(if needed)
    FDCAN_ACCEPT_IN_RX_FIFO0, // Accept extended messages in FIFO0
    FDCAN_REJECT_REMOTE,      // Reject remote frames
    FDCAN_REJECT_REMOTE);     // Reject remote frames
if (HAL_FDCAN_ConfigFilter(&hfdcan1, &sFilterConfig) != HAL_OK)
{
    /* Filter configuration Error */
    Error_Handler();
}
/* Start the FDCAN module */
if (HAL_FDCAN_Start(&hfdcan1) != HAL_OK) {
}
/* Start Error */
if (HAL_FDCAN_ActivateNotification(&hfdcan1,
FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0) != HAL_OK) {
}
/* Notification Error */
/* Configure Tx buffer message */

```

```
TxHeader.Identifier = 0x18FF1235;          // Example Extended ID
TxHeader.IdType = FDCAN_EXTENDED_ID;      // Use Extended ID
TxHeader.TxFrameType = FDCAN_DATA_FRAME;  // Set as a data frame
TxHeader.DataLength = FDCAN_DLC_BYTES_8;  // 8-byte data length
TxHeader.ErrorStateIndicator = FDCAN_ESI_ACTIVE;
TxHeader.BitRateSwitch = FDCAN_BRS_OFF;
TxHeader.FDFormat = FDCAN_CLASSIC_CAN;    // Use classic CAN
TxHeader.TxEventFifoControl = FDCAN_NO_TX_EVENTS;
TxHeader.MessageMarker = 0x00;

/*AAO-*/
/* USER CODE END WHILE */
/* USER CODE END FDCAN1_Init 2 */
}
```

RTOS

ODriveCanTask

- Read & send Odrives status odometry and status
 - Odometry
 - Twist with FK function
 - Position: integrating twist
 - Read periodic ODrive CAN status msgs
 - Send with Queue: CAN_2_UTX
- Process command from UartRX task
 - Receiving queue: URX_2_CAN
 - Using odrive.c functions:
 - Velocity CMD: Calculate required vel per motor
 - IK function
 - Config CMD: Send Odrive CAN Ctrl Msg

UartRxTask

- Rx data
- Parse
 - ↳ URX → CTRL
 - ↳ URX → ODrives
 - ↳ URX → UTX

UartTxTask

- Build msg
 - ↳ UEX → UTX
 - ↳ CTRL → UTX
 - ↳ ODrive → UTX
- Tx data

ROS2 SerialComm Node

- **UartRX:** Status / Odom / Sensor Data
 - ↳ Pub topics
- **UartTX:** Twist msg / Config
 - ↳ Sub to Twist topic
 - ↳ Sub to Config topic

Code Block Diagram (PENDIENTE)

OmniBasePositionCTRL_Attempt1 ← NOT YET TESTED

Code start 29/03/2025

Read ~~RAW~~ CNTs

Calc Δ CNTs

$\Theta_s \neq \Delta \text{CNTs} \times \text{Degs per CNT}$

$\Delta t = \text{current } t - \text{prev}$

$\omega_s = \Delta \text{CNT} \cdot \text{Rads per CNT}$

$Rx \Rightarrow \text{~~RAW~~ } x_d, y_d, \phi_d$

~~RAW~~ $e_x = x_d - x_{pos}; e_y = y_d - y_{pos}$

$\phi_d = \text{atan2}(e_y, e_x);$

$e_\phi = \phi_d - \phi_{pos} \leftarrow \text{IMU?} \leftarrow \text{Threshold}$

if e_ϕ bigger than threshold

//PID ~~position~~ angular position

$k_p e_\phi + k_i \int e_\phi dt + k_d \dot{e}_\phi \rightarrow \omega_d$

compute Necc $U_s(\omega_d, 0, 0);$

else

~~if~~ if // PID x position
 $k_p e_x + k_i \int e_x dt + k_d \frac{d}{dt} e_x \rightarrow \dot{x}_d$
 $\uparrow$
local

~~if~~ if
 $e_y > \text{thrshld}_y$ // PID y position
 $k_p e_y + k_i \int e_y dt + k_d \frac{d}{dt} e_y \rightarrow \dot{y}_d$
 $\uparrow$
local

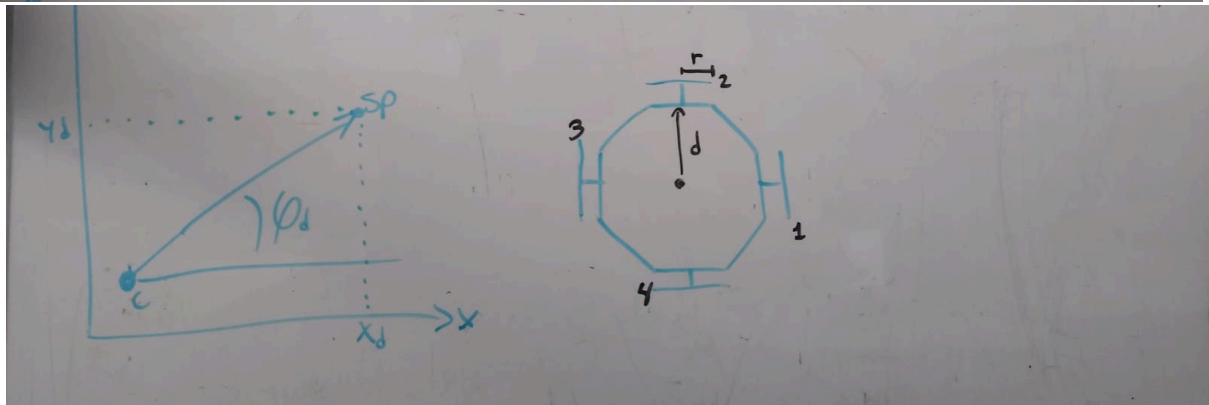
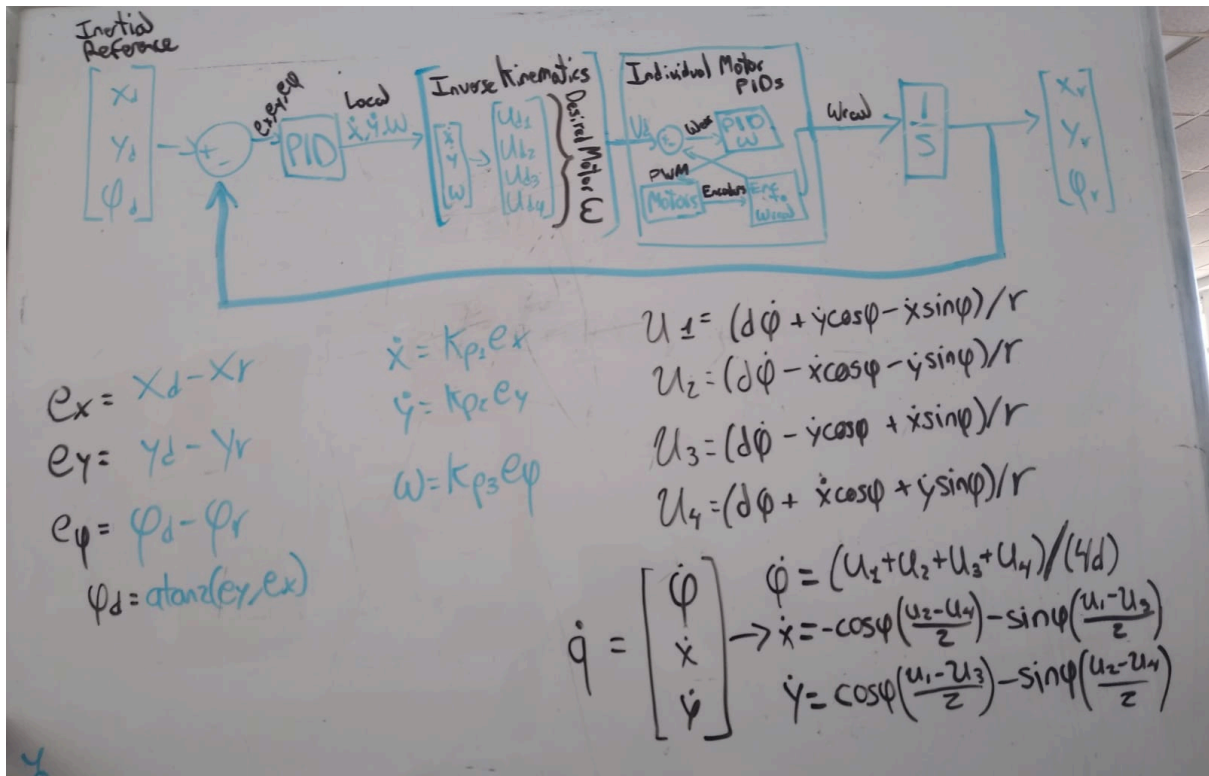
ComputeNewU_s($\phi, 0, \dot{y}_d, \dot{x}_d, d, u[i]$);

MotorErrors[] = ~~Need~~ Necessary U_s - W_s

~~M1~~ M₁ PWM = PID(Merr[0]);
M₂ PWM = PID(MWerr[1]);
M₃ PWM = PID(MWerr[2]);
M₄ PWM = PID(MWerr[3]); } limited to 100% duty cycle

Hol_Tim_Set_Compare(--- PWMs)... $\{q = \dot{x}, \dot{y}, \omega\}$
global speeds from U_s(ϕ, d, r, ω_s, q)

~~if~~
 $x_{pos} = \sum(\dot{x} \Delta t); y_{pos} = \sum(\dot{y} \Delta t); \phi = \sum(\dot{\phi} \Delta t)$



BNO055

Funcionó la BNO055 con la STM32H755 utilizando los archivos bno055.c, bno055.h y bno055_stm32.h de la siguiente librería:

https://github.com/ivyknob/bno055_stm32/tree/master

Fue necesario cambiar algunos nombres de macros de errores en los archivos .h porque estaban declarados con diferente nombre en las librerías correspondientes de stm32h7xx_hal.

En el código se obtienen ángulos de Euler o Cuaterniones con las siguientes funciones

```
bno055_vector_t BNO055_EulerVector = bno055_getVectorEuler();
```

```
bno055_vector_t BNO055_EulerVector = bno055_getVectorQuaternion();
```

Y son puestos en una estructura con la siguiente forma (w = 0 en ángulos de euler):

```
typedef struct {  
    double w;  
    double x;  
    double y;  
    double z;  
} bno055_vector_t;
```

Kinematics (PENDIENTE REDACTAR LIVESCRIPT)

Holonomic: movement constraints can be expressed solely in terms of position, if components move by same net amounts, final configuration is the same. Configuration space is known and independent of path taken.

Nonholonomic: constraints on velocity, e.g. a differential car cannot have “sideways” sliding velocity.

Modern Robotics, Chapter 2.4: Configuration and Velocity Constraints

Full Book MODERN ROBOTICS: MECHANICS, PLANNING, AND CONTROL

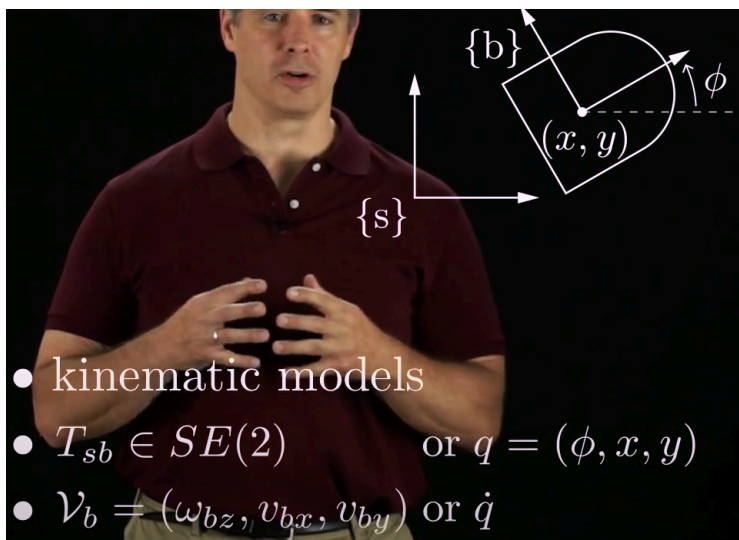
<https://hades.mech.northwestern.edu/images/7/7f/MR.pdf>

Chapter 13 Wheeled Mobile Robots (from same book ^^)

<https://modernrobotics.northwestern.edu/nu-gm-book-resource/13-1-wheeled-mobile-robots/#department>

<https://modernrobotics.northwestern.edu/nu-gm-book-resource/13-3-2-controllability-of-wheeled-mobile-robots-part-1-of-4/#department>

Analysis (PENDIENTE REDACTAR BIEN LO HECHO EN MATLAB)



Se tiene un marco global $\{s\}$ y un marco local $\{b\}$

La configuración del robot está dada por su posición y ángulo respecto al marco global

$$q = (\phi, x, y)$$

y su velocidad está dada por:

$$\dot{q} = (\dot{\phi}, \dot{x}, \dot{y})$$

Given a desired chassis velocity, what should the driving speeds of the wheels be?

$$H(0) = \begin{bmatrix} h_1(0) \\ \vdots \\ h_m(0) \end{bmatrix} \in \mathbb{R}^{m \times 3}$$

$$u = H(0)\mathcal{V}_b$$

wheel driving speed:

$$u_i = \underbrace{\frac{1}{r_i} [1 \quad \tan \gamma_i] \begin{bmatrix} \cos \beta_i & \sin \beta_i \\ -\sin \beta_i & \cos \beta_i \end{bmatrix} \begin{bmatrix} -y_i & 1 & 0 \\ x_i & 0 & 1 \end{bmatrix}}_{h_i(0), 1 \times 3 \text{ row vector}} \mathcal{V}_b$$

para cualquier llanta i

- Se tiene un centro de la llanta en una posición de (x_i, y_i) respecto al marco local $\{b\}$
 - La dirección en la que gira la llanta está a un ángulo β_i respecto a los ejes (x_i, y_i)
 - Los rodillos permiten movimiento a un ángulo γ_i respecto a la dirección $\perp$ a β_i .
- para una llanta omni $\gamma_i = 0$

En base a esas definiciones se puede calcular la velocidad de rotación del motor

$$u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = H(0)\mathcal{V}_b = \frac{1}{r} \begin{bmatrix} -d & 1 & 0 \\ -d & -1/2 & -\sin(\pi/3) \\ -d & -1/2 & \sin(\pi/3) \end{bmatrix} \begin{bmatrix} \omega_{bz} \\ v_{bx} \\ v_{by} \end{bmatrix}$$

<https://repository.umng.edu.co/server/api/core/bitstreams/97d7a947-b398-4d86-b3d6-63986f75ef82/content>

Memory usage investigation

Se investigó sobre el uso de memoria de la stm usando freertos, pude ver que estaba por default en 15KB total de heap para todas las tasks.

Aparentemente esto se declara en la sección de .bss (cosas no inicializadas, creo q puestas con 0 por default) del archivo .ld que define el uso de memoria de la stm. La sección .bss está puesta dentro de la memoria RAM_D1.

Investigando vi que puedo ver el uso de memoria con el build analyzer abriendo el archivo .map que se genera cada que se buildea un proyecto. Dado que no parecía estarse usando mucha memoria del RAM_D1, le modifiqué el heap size de FreeRTOSConfig.h a que fuera 64KB, esto esto debería evitar problemas con el stack por un tiempo, ya que ahora es bastante más.

Si fuese necesario usar más stack, podría aumentarse el tamaño del heap en FreeRTOSConfig.h y/o cambiar el heap_4.c por el heap_5.c del repositorio oficial de FreeRTOS para que el heap sea declarado utilizando secciones de memoria diferentes en diferentes memorias físicas (RAM_D1, RAM_D2, RAM_D3). Sería necesario investigar a profundidad cómo hacer bien esto si fuese necesario.

The screenshot shows an IDE with two main windows. The top window displays the FreeRTOSConfig.h file with various configuration macros. The bottom window shows the Build Analyzer output, including a table of memory regions.

FreeRTOSConfig.h Configuration:

```
63 #define configSUPPORT_STATIC_ALLOCATION 1
64 #define configSUPPORT_DYNAMIC_ALLOCATION 1
65 #define configUSE_IDLE_HOOK 0
66 #define configUSE_TICK_HOOK 0
67 #define configCPU_CLOCK_HZ ( SystemCoreClock )
68 #define configTICK_RATE_HZ ((TickType_t)1000)
69 #define configMAX_PRIORITIES ( 56 )
70 #define configMINIMAL_STACK_SIZE ((uint16_t)128)
71 #define configTOTAL_HEAP_SIZE ((size_t)(64 * 1024)) // ((size_t)15360)
72 #define configMAX_TASK_NAME_LEN ( 16 )
73 #define configUSE_TRACE_FACILITY 1
74 #define configUSE_16_BIT_TICKS 0
75 #define configUSE_MUTEXES 1
76 #define configQUEUE_REGISTRY_SIZE 8
77 #define configUSE_RECURSIVE_MUTEXES 1
78 #define configUSE_COUNTING_SEMAPHORES 1
79 #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
```

Build Console Output:

```
CDT Build Console [STM32H7_4_ENCODER_FREERTOS_CM7]
ar -r -o STM32H7_4_ENCODER_FREERTOS_CM7.elf STM32H7_4_ENCODER_FREERTOS_CM7.o
Finished building: default.size.stdout
Finished building: STM32H7_4_ENCODER_FREERTOS_CM7.1
19:28:05 Build Finished. 0 errors, 0 warnings. (too
```

Build Analyzer - Memory Regions Table:

Region	Start address	End address	Size	Free	Used	Usage (%)
RAM_D1	0x24000000	0x2407ffff	512 KB	440.91 KB	71.09 KB	13.88%
FLASH	0x08000000	0x0800ffff	1024 KB	859.41 KB	164.59 KB	16.07%
DTCMRAM	0x20000000	0x2001ffff	128 KB	128 KB	0 B	0.00%
RAM_D2	0x30000000	0x30047fff	288 KB	288 KB	0 B	0.00%
RAM_D3	0x38000000	0x3800ffff	64 KB	64 KB	0 B	0.00%
ITCMRAM	0x00000000	0x0000ffff	64 KB	64 KB	0 B	0.00%

Decent explanation on Static memory, Heap and Stack:

📺 Introduction to RTOS Part 4 - Memory Management | Digi-Key Electronics

Useful forum:

<https://forums.freertos.org/t/adjusting-heap-and-stack-size/12657>

Dualshock4 PS4 Bluetooth connection

La idea es conectar el Dualshock4 por bluetooth a un ESP32 y mandar la información del ESP32 por uart a la stm32.

Troubleshooting

1. **TAMAÑO DE STACK:** si alguna vez aparecen errores de inicialización de i2c o simplemente no jala nada, puede ser por el tamaño del stack de alguna task, que se llenó, me pasó muchas veces.

Resources

Núcleo STM32H755 Documentation

Datasheet: <https://www.st.com/resource/en/datasheet/stm32h755bi.pdf>

Reference Manual:

https://www.st.com/resource/en/reference_manual/rm0399-stm32h745755-and-stm32h747757-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

All STM32H7 docs:

https://www.st.com/en/microcontrollers-microprocessors/stm32h745-755/documentation.html?utm_source=chatgpt.com

God YT channels:

<https://www.youtube.com/@stm32world/videos>

<https://www.youtube.com/@ControllersTech>

MODERN ROBOTICS BOOK:

<https://hades.mech.northwestern.edu/images/7/7f/MR.pdf>

 Copia de InClassSPICANbus_AD2024

 Copia de QE_InClass_CANbus

TJA1051:

<https://pdf.utmel.com/r/datasheets/nxp-usainc-tja1051te118-datasheets-2574.pdf>

SCANF y PRINTF

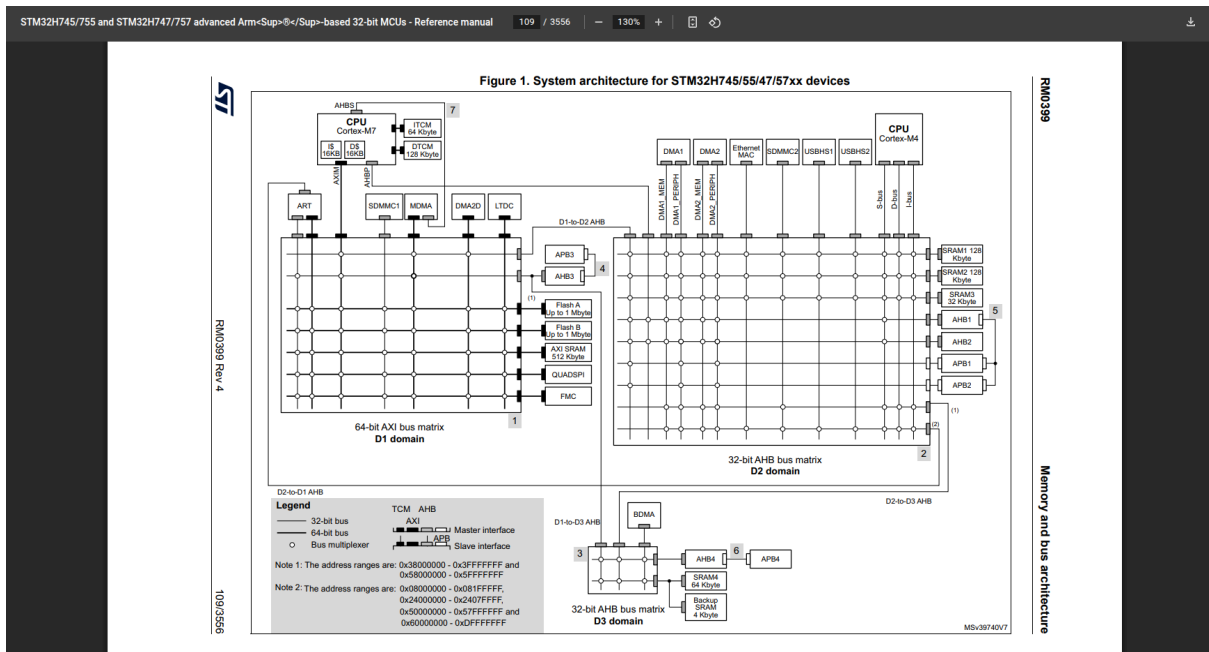
para ambos se deben activar las opciones de la sección properties → build → settings

Para printf basta con reescribir la función write en myprintf.h y myprintf.c

Para scanf también es lo mismo, pero se debe agregar esta línea al inicializar (después de hal_init()) jala):

```
setvbuf(stdin, NULL, _IONBF, 0);
```

System Architecture



Motor con encoder en Chaneque

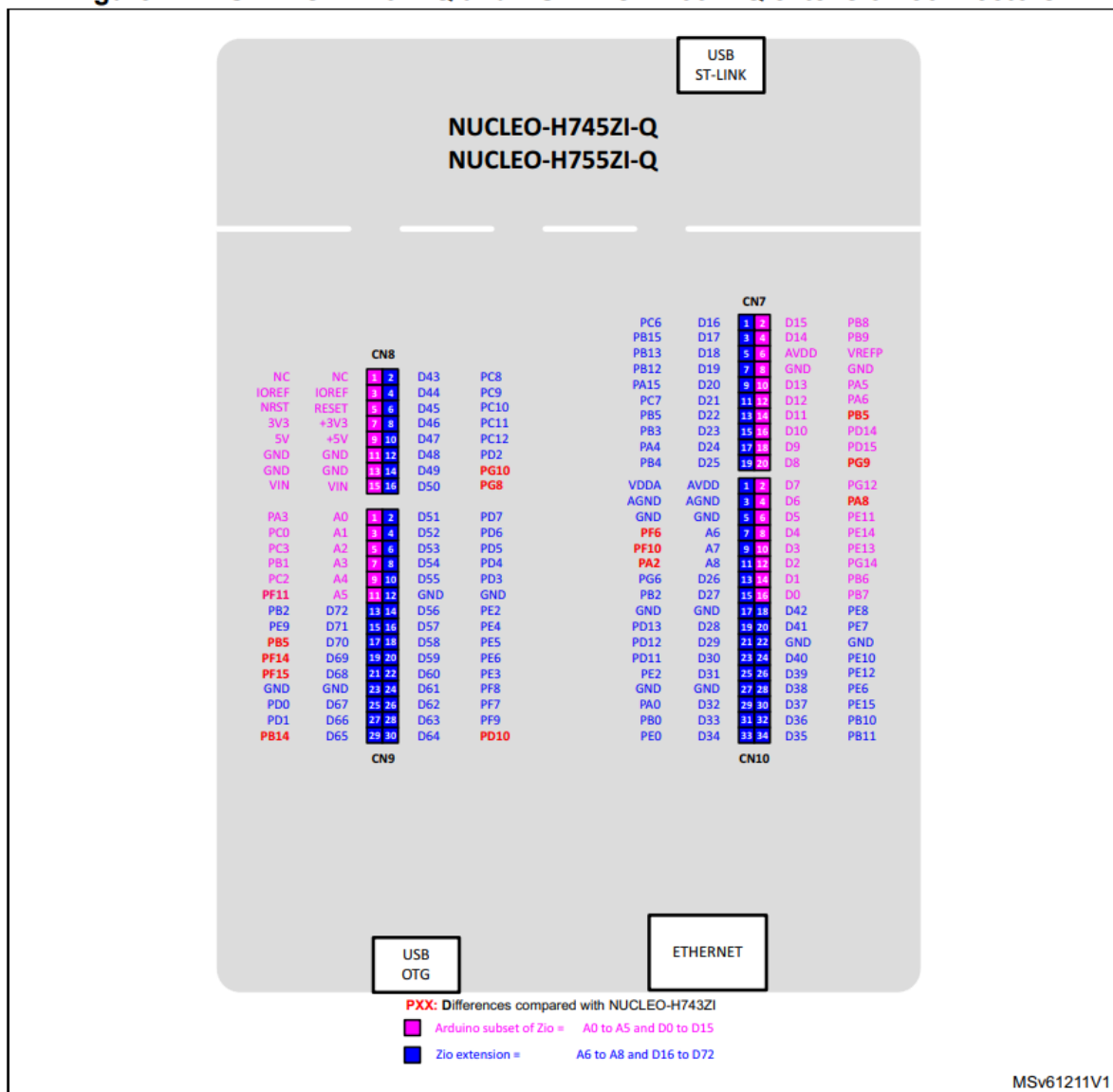
- Red:** Motor Power Supply Positive Pole + (switching can control forward and reverse rotation direction of the motor)
- Black:** Encoder Power Supply Negative Pole - (positive and negative poles 3.3-5V)
- Green:** Signal Feedback (11 signals when the motor take a turn)
- Yellow:** Signal Feedback (11 signals when the motor take a turn)
- Blue:** Encoder Power Supply Positive Pole + (positive and negative poles 3.3-5V)
- White:** Motor Power Supply Negative Pole - (switching can control forward and reverse rotation direction of the motor)

Type	AB Dual Phase Incremental Magnetic Hall Encoder		
Line Speed	Basic pulse 11PPRx gear reduction ratio		
Basic Function	Built-in pull-up shaping resistor, directly connected to the microcontroller		
Interface Type	PH2.0 (standard connecting cable)	Response Frequency	100KHz
Power Supply Voltage	DC3.3V / DC5.0V	Output Signal Type	Square wave AB phase
Basic Pulse Number	11PPR	Magnet Ring Trigger Class Quantity	22 poles (11 pairs of poles)

<https://roboticx.ps/product/dc-gear-motor-with-encoder/>

Female Headers Pinout

Figure 17. NUCLEO-H745ZI-Q and NUCLEO-H755ZI-Q extension connectors



UART Configuration

Note: the baud rate can be changed arbitrarily, as long as both ends have the same baud rate.

▼ Basic Parameters

Baud Rate	9600 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

▼ Advanced Parameters

Data Direction	Receive and Transmit
Over Sampling	16 Samples
Single Sample	Disable
ClockPrescaler	1
Fifo Mode	Disable
Txfifo Threshold	1 eighth full configuration
Rxfifo Threshold	1 eighth full configuration

▼ Advanced Features

Auto Baudrate	Disable
TX Pin Active Level Invers...	Disable
RX Pin Active Level Inver...	Disable
Data Inversion	Disable
TX and RX Pins Swapping	Disable
Overrun	Enable
DMA on RX Error	Enable
MSB First	Disable

Pin Name	Signal on Pin	Pin Context...	GPIO Outp...	GPIO mode	GPIO P...
PD8	USART3_TX	ARM Cortex-...	n/a	Alternate Fu...	No pull-u
PD9	USART3_RX	ARM Cortex-...	n/a	Alternate Fu...	No pull-u

Uart polling and interrupt

<https://deepbluembedded.com/how-to-receive-uart-serial-data-with-stm32-dma-interrupt-polling/>

<https://controllerstech.com/stm32-uart-3-receive-data-in-blocking-interrupt-mode/>

The image displays four sequential screenshots of the STM32H743 clock tree configuration, showing the flow of clock signals from input frequencies through various dividers and multiplexers to different system components.

RTC Clock Mux: Shows the input frequency of 32.769 KHz being divided by 32 to provide the RTC clock (32 KHz). It also shows the HSE input frequency of 4.50 MHz being divided by 1 to provide the HSE clock (4.50 MHz).

System Clock Mux: Shows the HSE input frequency of 4.50 MHz being divided by 1 to provide the HSE clock (4.50 MHz). It also shows the HSI input frequency of 12.288 MHz being divided by 1 to provide the HSI clock (12.288 MHz). The HSE clock is used to derive the system clock (SYSCLK) and the peripheral clock (PCLK).

SAI Clock Mux: Shows the HSE input frequency of 4.50 MHz being divided by 1 to provide the HSE clock (4.50 MHz). It also shows the HSI input frequency of 12.288 MHz being divided by 1 to provide the HSI clock (12.288 MHz). The HSE clock is used to derive the SAI clock (SAI1CLK) and the SAI2 clock (SAI2CLK).

USART Clock Mux: Shows the HSE input frequency of 4.50 MHz being divided by 1 to provide the HSE clock (4.50 MHz). It also shows the HSI input frequency of 12.288 MHz being divided by 1 to provide the HSI clock (12.288 MHz). The HSE clock is used to derive the USART clock (USART1CLK) and the USART2 clock (USART2CLK).

TIMERS

PWM

https://www.st.com/content/ccc/resource/training/technical/product_training/c4/1b/56/83/3a/a1/47/64/STM32L4_WDG_TIMERS_GPTIM.pdf/files/STM32L4_WDG_TIMERS_GPTIM.pdf/jcr:content/translations/en.STM32L4_WDG_TIMERS_GPTIM.pdf

Pag 54:

A few useful formulas 1/2

31

- PWM frequency set-up
 - Defined with auto-reload (ARR, in TIMx_ARR) and clock prescaler (PSC, in TIMx_PSC):
$$f_{PWM} = \frac{f_{TIM}}{(ARR+1) \times (PSC+1)}$$
 - Practically, one must start with PSC = 0 (no prescaler):
$$ARR = \frac{f_{TIM}}{f_{PWM} \times (PSC+1)} - 1 \rightarrow ARR = \frac{f_{TIM}}{f_{PWM}} - 1$$
 - If it yields a value above the 16-bit (or 32-bit) range, PSC must be increased until ARR fits:
$$ARR = \frac{f_{TIM}/2}{f_{PWM}} - 1 \rightarrow ARR = \frac{f_{TIM}/3}{f_{PWM}} - 1 \rightarrow ARR = \frac{f_{TIM}/4}{f_{PWM}} - 1 \rightarrow \dots$$



Pag 56:

A few useful formulas 2/2

32

- Duty cycle set-up
 - Defined with auto-reload (ARR, in TIMx_ARR) and compare values (PSC, in TIMx_CCRx):
$$Duty\ Cycle = \frac{CCRx+1}{ARR+1} \rightarrow CCRx = (Duty\ Cycle \times (ARR + 1)) - 1$$
- PWM resolution
 - The resolution gives the number of possible duty cycle values and indicates how fine the control on the PWM signal will be:
$$Res_{(steps)} = \frac{f_{TIM}}{f_{PWM}}$$
 - Another way of expressing it is in bits, as for giving a DAC converter output resolution:
$$Res_{(bits)} = \log_2\left(\frac{f_{TIM}}{f_{PWM}}\right)$$



50 Hz example

✓ Counter Settings

Prescaler (PSC - 16 bits v...	239
Counter Mode	Up
Counter Period (AutoRelo...	19999
Internal Clock Division (C...	No Division
auto-reload preload	Disable

✓ Clear Input

Clear Input Source	Disable
--------------------	---------

✓ PWM Generation Channel 1

Mode	PWM mode 1
Pulse (16 bits value)	0
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High

Encoder mode

▼ Counter Settings

Prescaler (PSC - 16 bits v... 0
Counter Mode Up
Counter Period (AutoRelo... 65535
Internal Clock Division (C... No Division
Repetition Counter (RCR -... 0
auto-reload preload Disable

▼ Trigger Output (TRGO) Paramet...

Master/Slave Mode (MSM... Disable (Trigger input effect not de...
Trigger Event Selection T... Reset (UG bit from TIMx_EGR)
Trigger Event Selection T... Reset (UG bit from TIMx_EGR)

▼ Encoder

Encoder Mode Encoder Mode TI1 and TI2

____ Parameters fo...

Polarity Rising Edge
IC Selection Direct
Prescaler Division Ratio No division
Input Filter 0

____ Parameters fo...

Polarity Rising Edge
IC Selection Direct
Prescaler Division Ratio No division
Input Filter 0